

AMATH 482: HOMEWORK 5 — FCN VS. CNN UNDER WEIGHT BUDGET CONSTRAINTS

ROHAN PANDEY

University of Washington, Seattle, WA
rpande@uw.edu

ABSTRACT. In this project, we investigate image classification on the Fashion-MNIST dataset using deep neural networks under strict parameter budget constraints. We implement and compare Fully Connected Networks (FCNs) and Convolutional Neural Networks (CNNs) with approximately 50K, 100K, and 200K parameters for FCNs, as well as CNN variants with budgets of roughly 10K, 20K, 50K, and 100K parameters. Our study includes extensive hyperparameter tuning, data augmentation, and regularization techniques to optimize performance while balancing accuracy and computational efficiency. In addition, we visualize training dynamics, intermediate feature maps, and synthesized class images to gain further insights into network behavior.

1. INTRODUCTION AND OVERVIEW

Image classification is a cornerstone task in computer vision, with recent advances driven by deep learning techniques. In this project, we explore two popular architectures for classifying images from the Fashion-MNIST dataset: Fully Connected Networks (FCNs) and Convolutional Neural Networks (CNNs). Unlike conventional methods, our approach enforces strict weight budget constraints—approximately 50K, 100K, and 200K parameters for FCNs, and similarly constrained CNN variants. This not only forces the models to learn compact representations but also provides insight into the trade-offs between model complexity, accuracy, and training efficiency. In addition to standard training and evaluation, we include visualization techniques to examine feature maps and generate synthesized images representative of each class using gradient-based optimization.

2. THEORETICAL BACKGROUND

Deep neural networks are designed to learn hierarchical representations of data through a series of linear transformations and non-linear activations. Two common architectures for image classification are Fully Connected Networks (FCNs) and Convolutional Neural Networks (CNNs).

Fully Connected Networks (FCNs). FCNs consist of layers in which every neuron is connected to every neuron in the subsequent layer. While this design is straightforward, it can lead to a large number of parameters. Under a strict weight budget, the number of hidden units must be carefully selected. In an FCN, an input image is first flattened into a vector (e.g., a 28x28 image becomes a 784-dimensional vector). Each layer computes a linear transformation:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

followed by a non-linear activation function such as the Rectified Linear Unit (ReLU):

$$\text{ReLU}(x) = \max(0, x).$$

For classification tasks, the final layer outputs logits that are then transformed into a probability distribution using the softmax function:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}},$$

where C is the number of classes. The network is trained by minimizing the Cross-Entropy Loss:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\sigma(\mathbf{z})_i),$$

which quantifies the dissimilarity between the predicted probability distribution and the true labels [1].

Convolutional Neural Networks (CNNs). CNNs are specifically designed to process spatial data like images. Convolutional layers apply learnable filters to local regions of the input, capturing spatial patterns while drastically reducing the number of parameters compared to FCNs. The convolution operation is defined as:

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_m \sum_n \mathbf{I}(i - m, j - n) \mathbf{K}(m, n),$$

where $\mathbf{I}$ is the input image and $\mathbf{K}$ is the convolution kernel. In addition to the kernel size, two important parameters in convolutional layers are the **stride** and **padding**.

Stride refers to the step size with which the convolutional kernel moves across the input image. A stride of 1 means the filter is applied at every possible location, while a larger stride results in a down sampled output feature map.

Padding involves adding extra border pixels (commonly zeros) around the input image. This can help control the spatial dimensions of the output feature maps. For instance, *same* padding is used to preserve the input dimensions, whereas *valid* padding (no padding) typically reduces the output size. These mechanisms are essential for designing deep architectures with controlled feature map sizes [1].

Pooling layers (e.g., max pooling) further reduce the spatial dimensions, leading to a compact representation that is subsequently fed into fully connected layers. CNNs typically employ techniques such as batch normalization and dropout to improve training stability and prevent overfitting [3].

Regularization and Weight Initialization. Regularization techniques are essential to mitigate overfitting, particularly in high-capacity models. **Dropout** randomly deactivates a fraction of neurons during training, promoting redundancy and reducing reliance on specific neurons. In addition, robust weight initialization is crucial for effective training. Methods such as Xavier (Glorot) initialization and Kaiming (He) initialization maintain consistent variances of activations across layers, preventing issues like vanishing or exploding gradients [2]. These strategies, combined with parameter budget constraints, encourage the network to develop efficient and compact representations while still achieving high performance.

3. ALGORITHM IMPLEMENTATION AND DEVELOPMENT

Below is a high-level algorithm outlining the main steps of our project.

Algorithm 1 Training and Evaluation of FCN and CNN Models under Weight Budgets

```

1: Input: Fashion-MNIST dataset, parameter budgets, hyperparameters (learning_rate,
   epochs, batch_size)
2: Output: Trained models and performance metrics
3: /* Data Loading and Preprocessing */
4: Download and normalize the Fashion-MNIST dataset.
5: Split the dataset into Training, Validation, and Test sets.
6: Apply data augmentation (e.g., random flip, rotation) to the training set.
7: /* Model Definition */
8: Define FCN architectures with  $\sim 50K$ ,  $\sim 100K$ , and  $\sim 200K$  parameters.
9: Define CNN architectures with  $\sim 10K$ ,  $\sim 20K$ ,  $\sim 50K$ , and  $\sim 100K$  parameters.
10: /* Training */
11: for each model variant do
12:   for epoch = 1 to epochs do
13:     Train the model using mini-batch gradient descent.
14:     Update weights via backpropagation.
15:     Compute and log training loss and validation accuracy.
16:   end for
17:   Evaluate the trained model on the Test set.
18: end for
19: /* Visualization and Reporting */
20: Plot training and validation loss and accuracy curves.
21: Visualize intermediate feature maps for CNNs.
22: Generate synthesized class images using gradient ascent.

```

COMPUTATIONAL RESULTS

In this section, we present the computational results obtained from our experiments. We first analyze the training dynamics via the loss and accuracy curves (see Figures 1, 2), which provide insight into the convergence behavior and stability of the models over the training epochs. Notice that there are 2 graphs, one for FCN and one for CNN.

Next, Table 1 summarizes the final performance metrics for each model variant, including training loss, training accuracy, validation loss, validation accuracy, as well as test loss and test accuracy.

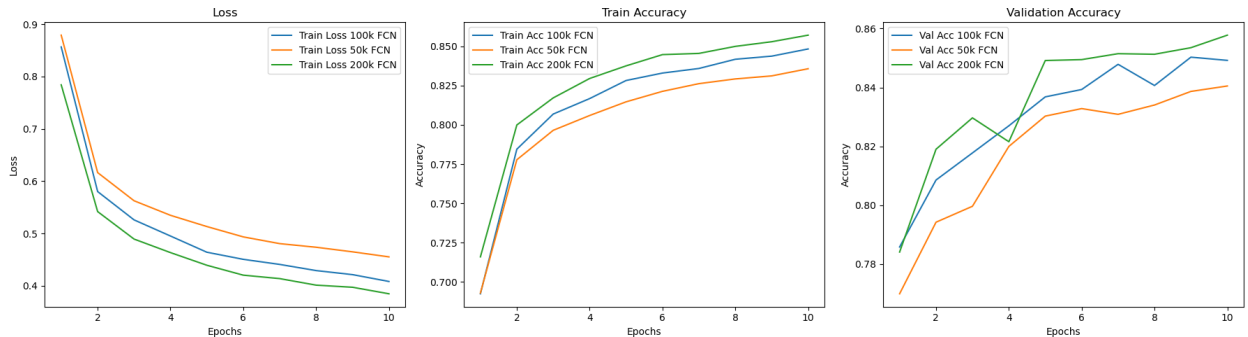


FIGURE 1. Training loss and accuracy curves for FCN models

In the above figure, we have 3 plots of the Loss curve, Training Accuracy, and Validation Accuracy for 50K, 100K, and 200K FCN models. Overall in all three the general trend seems to be the same,

that the 200K model has the lowest loss and highest accuracy, followed by the 100K model, and then the 50K model.

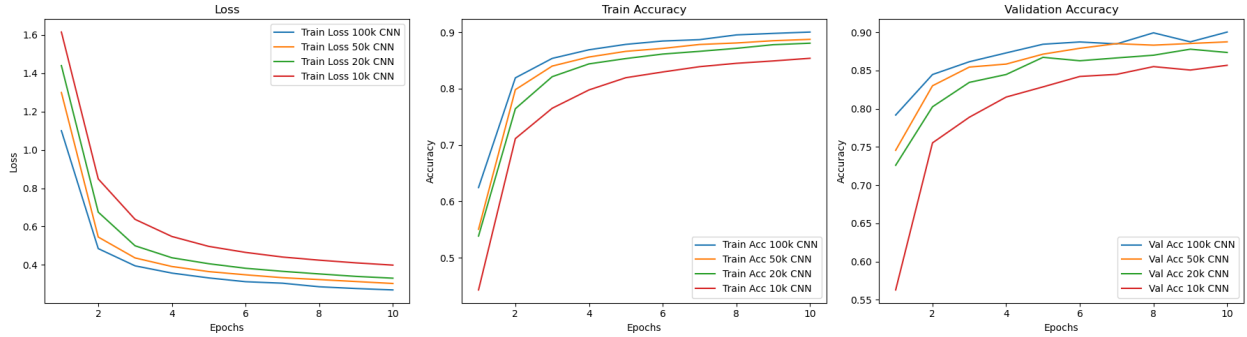


FIGURE 2. Training loss and accuracy curves for CNN models

Coming into the CNN model, we have 4 plots of the Loss curve, Training Accuracy, and Validation Accuracy for 10K, 20K, 50K, and 100K CNN models. The trend is similar to the FCN model, where the 100K model has the lowest loss and highest accuracy, followed by the 50K model, then the 20K model, and finally the 10K model. However, comparatively, although more data is available in 1, we can notice the sharper decline in the loss graph, and sharper increase in the accuracy graph for the CNN models.

TABLE 1. Summary of Final Training, Validation, and Test Performance

Model	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)	Test Loss	Test Acc (%)
FCN 50K	0.4554	83.57	0.4443	84.05	0.4077	85.73
FCN 100K	0.4084	84.83	0.4059	84.92	0.3773	86.37
FCN 200K	0.3849	85.71	0.3924	85.78	0.3596	87.05
CNN 10K	0.3990	85.39	0.3939	85.69	0.4036	85.49
CNN 20K	0.3308	88.06	0.3440	87.37	0.3262	88.74
CNN 50K	0.3031	88.74	0.3056	88.76	0.2953	89.29
CNN 100K	0.2696	90.04	0.2577	90.04	0.2565	90.58

Analysis. Table 1 presents the key performance metrics across all model variants. A clear trend is observed for the Fully Connected Networks (FCNs): as the parameter budget increases from approximately 50K to 200K, both the training and test accuracies show modest improvements (from 85.73% to 87.05% test accuracy). This indicates that while additional parameters allow for learning more complex representations, the improvements are incremental.

On the other hand, the Convolutional Neural Networks (CNNs) demonstrate more significant performance gains. The CNN 100K model achieves the highest test accuracy of 90.58%, suggesting that the convolutional architecture is particularly well-suited for image data. Even lower-capacity CNN variants, such as the CNN 10K model, perform competitively, although with lower accuracies (85.49%). In summary, the results indicate that CNN architectures not only converge faster but also generalize better compared to FCNs, especially as the parameter budget increases.

Further analysis of the training curves (loss and accuracy plots) will help illuminate the convergence behavior of each model and provide additional insight into the trade-offs between model complexity and performance.

SUMMARY AND CONCLUSIONS

In this project, we investigated image classification on the Fashion-MNIST dataset using Fully Connected Networks (FCNs) and Convolutional Neural Networks (CNNs) under strict weight budget constraints. Our results demonstrate that while FCNs can achieve competitive performance with limited parameters, CNNs outperform FCNs across all budget levels. The CNN 100K model achieved the highest test accuracy of 90.58%, highlighting the effectiveness of convolutional architectures for image classification tasks. The training dynamics and performance metrics provide valuable insights into the trade-offs between model complexity, accuracy, and computational efficiency. Future work could explore additional regularization techniques, network architectures, and hyperparameter tuning to further improve performance under weight budget constraints.

ACKNOWLEDGEMENTS

The author is thankful to Professor Frank for providing the materials and resources needed to complete this assignment. The author is also thankful to the peers Eric Ye and Brighton for discussion on implementing Depthwise Convolution and pseudocode for this project. The author would also like to acknowledge that the code for this project, as well as the Abstract, Introduction and Overview, as well as the theoretical background sections were heavily derived from the Authors own work in Homework 4 of the AMATH 482 course. Lastly, the author would also like to acknowledge that many of the code including building the Neural Network has been discussed in the author's research group since October 2024, as the author is part of the WXML Group on Backdooring ML Models and one of the ML models built was to classify images using a Neural Network, with the dataset being the Fashion-MNIST and MNIST datasets. The research project already involved building CNNs and the author has used inspiration from that project to build the CNNs in this project.

REFERENCES

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.
- [3] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.